LaTeX Note Template

Paper Author: Alpha, Beta, Gamma

Journal: Rubbish and Garbage

Published: 2025-11-27 **DOI:** 10.1234/journal.2025.12345

Author: Chen Huaneng **Email:** huanengchen@foxmail.com **Date:** 2025-11-27

1 表格展示

1.1 表格内脚注和跨页表格

表 1: mVRP-D 模型符号及含义

符号	含义
$G = (N, E)$	定义顾客和仓库节点的图
$N = \{0, 1, 2, \ldots, n, n+1\}$	图的节点，包括顾客节点 $i, j \in N_c = \{1, 2, \ldots, n\}$ 以及仓库节点（其中一个是虚拟仓库）$\{0, n+1\}$
$N_c = N \setminus \{0, n+1\}$	顾客节点集合 $\{1, 2, \ldots, n\}$
$N_0 = N \setminus \{n+1\}$	无人机或卡车可能的出发节点集合 $\{0, 1, \ldots, n\}$
$N_+ = N \setminus \{0\}$	无人机或卡车可能的到达节点集合 $\{1, 2, \ldots, n, n+1\}$
E	图的弧，可能的配送路径
q_i	顾客 i 的需求（包裹重量）
K	卡车组的集合，卡车组的数量为 $\mid K \mid$，卡车组的索引为 k
f	无人机空载时的自重
m_k^K	卡车最大载重限制
m_k^D	卡车 k 上的无人机的最大载重限制
e_k^D	卡车组 k 的无人机空载时的最大续航里程
e_{ki}^D	卡车 k 上的无人机载重顾客 i 的需求时的续航里程，当 $q_i \le m_k^D$ 时，$e_{ki}^D = e_k^D \frac{f}{f+q_i}$，否则 $e_{ki}^D = 0$[a]
t_{ijk}^K	卡车 k 从节点 i 到节点 j 所需的行驶时间
t_{ijk}^D	卡车 k 上的无人机从节点 i 到节点 j 所需的飞行时间
$\langle i, j, h \rangle$	无人机的运输路径，从 i 节点起飞，配送顾客节点 j，最终和卡车在节点 h 会合
F	所有可能的无人机的运输路径，即无人机搭载顾客节点 j 的包裹时的最大续航里程不小于从 i 节点到 j 顾客节点和从 j 顾客节点到卡车所在节点 h 的飞行时间之和 $(i, j, h) \in F, i \in N_0, j \in \{N_c : j \ne i\}, h \in \{N_+ : h \ne j, h \ne i, t_{ijk}^D + t_{jhk}^D \le e_{kj}^D\}$

Continued on next page

表 1: mVRP-D 模型符号及含义 (Continued)

符号	含义
s^K	单辆卡车的单位时间内的单位运输成本
s^D	单架无人机单位时间内的单位运输成本
s^W	单辆卡车的单位时间内的单位等待时间成本
s^G	单个卡车组的固定使用成本
M	足够大的正数
α_{ijk}	二元决策变量，当卡车 k 从节点 $i \in N_0$ 行驶到节点 $j \in N_+$ 时等于 1，否则为 0
β_{ijhk}	二元决策变量，当卡车 k 上的无人机从节点 $i \in N_0$ 起飞，服务顾客节点 $j \in N_c$，最终和卡车 k 在节点 $h \in N_+$ 会合时等于 1，否则等于 0
μ_{ik}	整数，表示卡车 k 的路径中节点 i 的次序
γ_{ijk}	二元决策变量，当卡车 k 服务节点 i 的次序在服务节点 j 的次序之前时等于 1，否则等于 0
τ_{ik}^K	非负连续变量，卡车 k 到达节点 i 的时间
τ_{ik}^D	非负连续变量，卡车 k 上的无人机到达节点 i 的时间
ρ_{ik}	非负连续变量，卡车组 k 到达节点 i 的时间（即卡车或无人机最后一个到达节点 i 的时间）
ε_k	二元变量，当卡车组 k 被选中时等于 1（即卡车组 k 参与了配送），否则等于 0

[a] 文章中假设无人机的最大续航里程与无人机的自重和运载的顾客包裹的重量之和成反比。

1.2 单元格内换行

双行 表头	双行 表头
简单 粗暴	ABCD EF
	更大的竖直空距

1.3 跨行跨列表格

A Text!	ABC	DEF
	abc	def
Nothing		XYZ xyz

1.4 嵌套表格

<table>
<tr><td>a</td><td colspan="2">bbb</td><td>c</td></tr>
<tr><td rowspan="2">a</td><td>a</td><td>b</td><td rowspan="2">c</td></tr>
<tr><td>aa</td><td>bb</td></tr>
<tr><td>a</td><td colspan="2">b</td><td>c</td></tr>
</table>

2 参考文献、数学公式

2.1 Traveling Salesman Problem

旅行商问题（Traveling Salesman Problem, TSP）是组合优化领域的经典问题之一，其核心目标是给定城市列表和每对城市之间的距离，求恰好访问每个城市一次并返回起始城市的最短可能路线。该问题于 1930 年正式提出，是优化中研究最深入的问题之一，被用作许多优化方法的基准。自从该问题被正式提出以来，一直是运筹学、计算机科学和物流管理等领域的研究热点，尽管该问题在计算上很困难，但许多启发式方法和精确算法是已知的[1-2]。

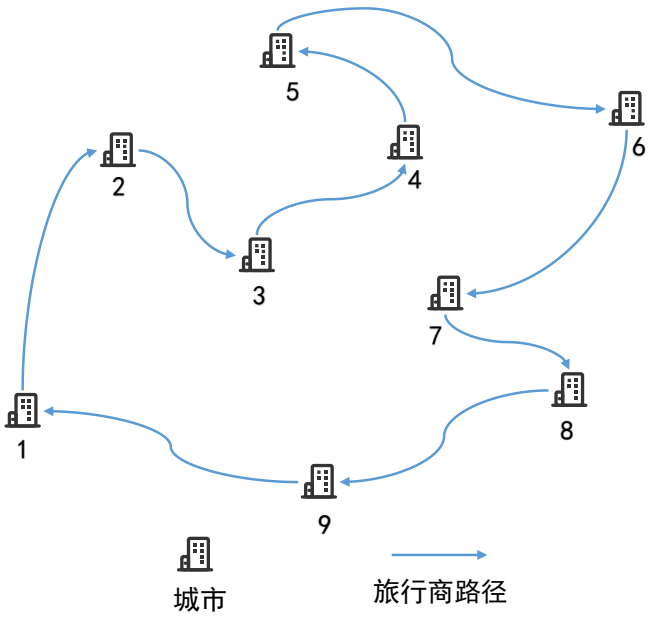


图 1: TSP 示意图

TSP 可以表述为整数线性规划模型[3]：假设共有 N 个城市，每个城市的编号为 $1,\cdots,N$，从城市 i 到城市 j 的旅行成本（距离）为 $c_{ij}>0$。旅行商的目标是从任意一个城市出发访问完所有的城市，每个城市只能访问一次，最后回到最初的城市，目标是找到一条依次访问所有城市且访问城市不重复的最短路线。TSP 中的决策变量为 $x_{ij}=\begin{cases}1, & \text{存在从城市 } i \text{ 到城市 } j \text{ 的路径}\\ 0, & \text{其他}\end{cases}$，城市节点集合表示为 $V(|V|=N)$。由于可能存在子回路，所以在构建 TSP 模型时需要消除会产生子回路的情况，这里采用 Miller-Tucker-Zemlin (MTZ) 约束进行子回路的消除[4]，引入连续变量 $u_i(\forall i\in V, u_i\geq 0)$，其取值可以为任何非负实数（实数集合表示为 R）。这里用 u_i 表示编号为 i 的城市的访问次序，比如当 $u_i=5$ 时表示编号为 1 的城市是从出发点开始，第 5 个被访问到的点。因此，TSP 的数学模型可以

表示为 (1)-(6)。

$$\min \quad \sum_{i \in V} \sum_{j \in V, i \neq j} c_{ij} x_{ij} \tag{1}$$

$$\text{s.t.} \quad \sum_{i \in V} x_{ij} = 1, \qquad \forall j \in V, i \neq j \tag{2}$$

$$\sum_{j \in V} x_{ij} = 1, \qquad \forall i \in V, i \neq j \tag{3}$$

$$u_i - u_j + N x_{ij} \leq N - 1, \qquad \forall i, j \in V; i \neq j \tag{4}$$

$$u_i \geq 0, \qquad u_i \in R \tag{5}$$

$$x_{ij} \in \{0, 1\}, \qquad i, j \in V; i \neq j \tag{6}$$

目标函数(1)表示最小化访问所有城市的成本（距离），约束(2)和(3)保证每个城市节点的入度和出度为 1，即每个城市只进入一次和出去一次，保证了每个城市只访问一次，不会被重复访问，约束(4)消除子回路，约束(5)和(6)表示变量的取值范围。

3 颜色

3.1 颜色名称测试（dvipsnames, svgnates, x11names）

- Red (dvipsname)
- DeepSkyBlue2 (x11names)
- ForestGreen (dvipsname)
- DarkOrange1 (x11names)

3.2 自定义颜色示例

这是自定义的蓝色文本。

这是自定义的黄色文本。

3.3 颜色混合示例

混合 50% 蓝色和 50% 红色

混合 60% 绿色和 40% 白色

混合 30% 橙色、70% 黄色

3.4 背景色示例

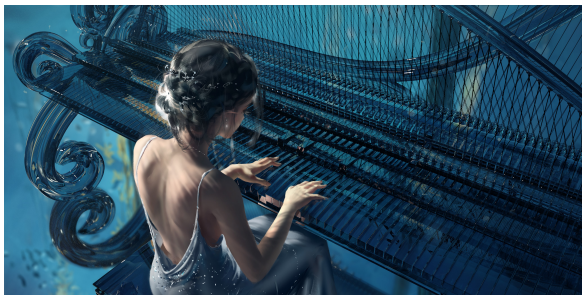
这是 SkyBlue 背景色的文本。

4 插入图片及子图

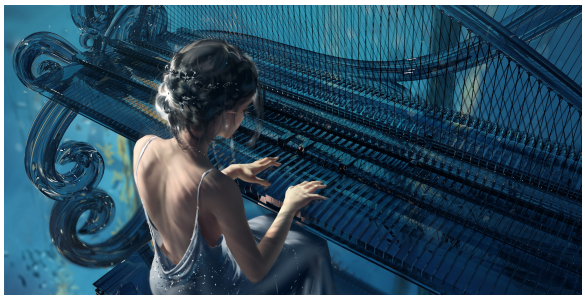
4.1 插入图片

图 2: 图片示例

4.2 子图排列



(a) 子图 A



(b) 子图 B

(c) 子图 C

(d) 子图 D

图 3: 多张子图排列示例 1

(a) 子图 1

(b) 子图 2

(c) 子图 3

图 4: 多张子图排列示例 2

5 伪代码

Algorithm 1: what

Input: This is some input

Output: This is some output

/ This is a comment */*

some code here
$x \leftarrow 0$
$y \leftarrow 0$
if $x > 5$ **then**
    x is greater than 5 *// This is also a comment*
else
    x is less than or equal to 5
end if
foreach *y in 0..5* **do**
    $y \leftarrow y + 1$
end foreach
for *y in* 0..5 **do**
    $y \leftarrow y - 1$
end for
while $x > 5$ **do**
    $x \leftarrow x - 1$
end while
return Return something here

6 代码环境

6.1 多行代码

Python 代码:

```python
def hello_en():
    print("Hello, world!")

def hello_cn():
    print(" 你好, 世界! ")

hello_en()
hello_cn()
```

C++ 代码:

```cpp
#include <iostream>
#include <vector>
#include <string>

using namespace std;

void getNext(int *next, const string &p) {
```

```cpp
    int m = p.size(), j = 0; // j is the length of the previous longest prefix suffix
    next[0] = 0; // The first character has no proper prefix or suffix
    for (int i = 1; i < m; ++i) { // Start from the second character
        // Check if the j > 0 because we need to index to the previous next value
        while (j > 0 && p[i] != p[j]) { // Backtrack if there is a mismatch
            j = next[j - 1]; // j - 1 because next is 0-indexed
        }
        if (p[i] == p[j]) { // If characters match, increment j, namely the length of the current prefix
            ++j;
        }
        next[i] = j; // Set the next value for the current character
    }
}

int kmp(const string &s, const string &p) {
    int n = s.size(), m = p.size();
    if (n < m) { // Check if the pattern is longer than the text
        cout << "Pattern is longer than text." << endl;
        return -1;
    }
    if (m == 0) { // Check if the pattern is empty
        cout << "Empty pattern." << endl;
        return -1;
    }
    vector<int> next(m);
    getNext(&next[0], p); // Initialize the next array for the pattern p
    for (int i = 0, j = 0; i < n; ++i) { // i is the index in the main string s and j is the index in the pattern p
        while (j > 0 && s[i] != p[j]) { // Mismatch after j matches
            j = next[j - 1]; // Use the next array to skip unnecessary comparisons, minus 1 because next is 0-indexed
        }
        if (s[i] == p[j]) { // Match found and increment j to check next character
            ++j;
        }
        if (j == m) { // If j equals the length of the pattern, the first occurrence is found
            cout << "Pattern found at index: " << i + 1 - m << endl;
            return 0; // Return after finding the first occurrence
        }
    }
    return -1; // If no match is found, return -1
}

int main() {
    string s, p;
    cin >> s >> p;
    cout << "The main string is: " << s << endl;
    cout << "The pattern string is: " << p << endl;
    int result = kmp(s, p);
    if (result == -1) {
        cout << "Pattern not found." << endl;
    } else {
        cout << "Pattern found successfully." << endl;
    }
    return 0;
}
```

6.2 行内代码

This is an example of minted in the same line `print("Hello world!")`.

7 编辑体验

7.1 超链接

The url (outer link) color is just like this. The inner link color is just like this one → 7.1. The citation is like this[1].

7.2 脚注

This is a footnote example[1].

7.3 边注

A marginpar.

Such a marginpar is like that. This is after the marginpar.

Hello world!

7.4 智能交叉引用

这是一个数学公式引用：公式 3，这是一个图片引用：图 3a，这是一个表格引用：表 1。

8 Section example

8.1 Subsection example

This is a subsection example.

8.1.1 Subsubsection example

This is a subsubsection example.

Unnumbered section example

Unnumbered subsection example

This is a unnumbered subsection example.

Unnumbered subsubsection example

This is a unnumbered subsubsection example.

9 中文字体测试

9.1 中文主字体（LXGWWenKaiScreen）

这是默认的中文字体（加粗和*斜体*分别为方正黑体和方正楷体）。

加粗文本示例：这是一段加粗的中文文本。

斜体文本示例：这是一段斜体的中文文本。

[1] Related footnote is here.

9.2 中文无衬线字体（FZHTJW）

这是无衬线字体（**加粗**效果由 AutoFake 生成，*斜体*为方正仿宋）。

加粗文本示例：这是一段加粗的无衬线中文文本。

斜体文本示例：这是一段斜体的无衬线中文文本。

9.3 等宽字体（LXGWWenKaiMonoScreen）

这是等宽字体（**加粗**效果由 AutoFake 生成，*斜体*为方正书宋）。

加粗文本示例：这是一段加粗的等宽中文文本。

斜体文本示例：这是一段斜体的等宽中文文本。

9.4 混合字体测试

主字体（**LXGWWenKaiScreen**）

无衬线字体（FZHTJW）

等宽字体（LXGWWenKaiMonoScreen）

主字体和无衬线的切换效果。

等宽字体与**普通字体**的对比。

10 English Font Test

10.1 English Main Font (Times New Roman)

This is the default English font (serif).

Bold and *Italic* effects are generated automatically.

10.2 English Sans-serif Font (Arial)

This is the English sans-serif font.

Bold and *Italic* effects are generated automatically.

10.3 English Monospace Font (Consolas)

```
This is the English monospace font.

Bold and Italic effects are generated automatically.
```

10.4 中英文混合字体测试

Main Font: Times New Roman + **中文主字体:** LXGWWenKaiScreen.

Sans-serif Font: Arial + **中文无衬线字体**: FZHTJW.

Monospace Font: Consolas + **中文等宽字体:** LXGWWenKaiMonoScreen.

11 滕王阁序

豫章故郡，洪都新府。星分翼轸 (zhěn)，地接衡庐。襟三江而带五湖，控蛮荆而引瓯（ōu）越。物华天宝，龙光射牛斗之墟；人杰地灵，徐孺下陈蕃 (fān) 之榻。雄州雾列，俊采星驰，台隍 (huáng) 枕夷夏之交，宾主尽东南之美。都督阎公之雅望，棨 (qǐ) 戟遥临；宇文新州之懿 (yì) 范，襜 (chān) 帷 (wéi) 暂驻。十旬休假，胜友如云；千里逢迎，高朋满座。腾蛟起凤，孟学士之词宗；紫电清霜，王将军之武库。家君作宰，路出名区；童子何知，躬逢胜饯。

时维九月，序属三秋。潦（lǎo）水尽而寒潭清，烟光凝而暮山紫。俨 (yǎn) 骖騑 (cān fēi) 于上路，访风景于崇阿 (ē)。临帝子之长洲，得天人之旧馆。层峦耸翠，上出重霄；飞阁流（一作翔）丹，下临无地。鹤汀（tīng）凫 (fú) 渚（zhǔ），穷岛屿之萦 (yíng) 回；桂殿兰宫，即（一作列）冈峦之体势。

披绣闼（tà），俯雕甍 (méng)。山原旷其盈视，川泽纡 (yū) 其骇瞩。闾 (lǘ) 阎 (yán) 扑地，钟鸣鼎食之家；舸（gě) 舰弥津，青雀黄龙之舳（zhú）。云销雨霁 (jì)，彩彻区明（或作虹销雨霁，彩彻云衢 qú）。落霞与孤鹜 (wù) 齐飞，秋水共长天一色。渔舟唱晚，响穷彭蠡（l ǐ）之滨；雁阵惊寒，声断衡阳之浦。

遥襟甫畅，逸兴遄 (chuán) 飞。爽籁发而清风生，纤歌凝而白云遏 (è)。睢 (suī) 园绿竹，气凌彭泽之樽；邺 (yè) 水朱华，光照临川之笔。四美具，二难并。穷睇眄 (dì miǎn) 于中天，极娱游于暇日。天高地迥 (jiǒng)，觉宇宙之无穷；兴尽悲来，识盈虚之有数。望长安于日下，目吴会（kuài）于云间。地势极而南溟 (míng) 深，天柱高而北辰远。关山难越，谁悲失路之人；萍水相逢，尽是他乡之客。怀帝阍 (hūn) 而不见，奉宣室以何年。

嗟 (jiē) 乎！时运不齐，命途多舛 (chuǎn)；冯唐易老，李广难封。屈贾谊（yì）于长沙，非无圣主；窜梁鸿于海曲，岂乏明时？所赖君子见机，达人知命。老当益壮，宁移白首之心？穷且益坚，不坠青云之志。酌贪泉而觉爽，处涸辙（hé zhé）以犹欢。北海虽赊（shē），扶摇可接；东隅 (yú) 已逝，桑榆非晚。孟尝高洁，空余报国之情；阮籍猖狂，岂效穷途之哭！

勃，三尺微命，一介书生。无路请缨，等终军之弱冠（guàn）；有怀投笔，慕宗悫（què）之长风。舍簪（zān）笏（hù）于百龄，奉晨昏于万里。非谢家之宝树，接孟氏之芳邻。他日趋庭，叨 (tāo) 陪鲤对；今兹捧袂 (mèi)，喜托龙门。杨意不逢，抚凌云而自惜；钟期既遇，奏流水以何惭？

呜呼！胜地不常，盛筵 (yán) 难再；兰亭已矣，梓 (zǐ) 泽丘墟。临别赠言，幸承恩于伟饯；登高作赋，是所望于群公。敢竭鄙怀，恭疏短引；一言均赋，四韵俱成。请洒潘江，各倾陆海云尔。

滕王高阁临江渚，佩玉鸣鸾罢歌舞。

画栋朝飞南浦云，珠帘暮卷西山雨。

闲云潭影日悠悠，物换星移几度秋。

阁中帝子今何在？槛外长江空自流。

12 Do Not Go Gentle into That Good Night

Do not go gentle into that good night,

Old age should burn and rave at close of day;

Rage, rage against the dying of the light.

Though wise men at their end know dark is right,

Because their words had forked no lightning they

Do not go gentle into that good night.

Good men, the last wave by, crying how bright

Their frail deeds might have danced in a green bay,

Rage, rage against the dying of the light.

Wild men who caught and sang the sun in flight,

And learn, too late, they grieved it on its way,

Do not go gentle into that good night.

Grave men, near death, who see with blinding sight

Blind eyes could blaze like meteors and be gay,

Rage, rage against the dying of the light.

And you, my father, there on the sad height,

Curse, bless, me now with your fierce tears, I pray.

Do not go gentle into that good night.

Rage, rage against the dying of the light.

13 Theorem, Proposition, Proof

> **Theorem 13.1.** *If $1 < p < \infty$ and $m > n/p$, or $p = 1$ and $m \geq n$, there exist a constant $C = C(m, n, \gamma, p)$, such that*
>
> $$\|R^m u\|_{L^\infty(\Omega)} \leq C d^{m-n/p} |u|_{W_p^m(\Omega)}$$
>
> *for all $u \in W_p^m(\Omega)$.*

Proof. First, we assume that $u \in C^m(\Omega) \cap W_p^m(\Omega)$. We can use the pointwise representation of $R^m u(x)$.

$$\begin{aligned} |R^m u(x)| &= m \left| \sum_{|\alpha|=m} \int_{C_x} k_\alpha(x,z) D^\alpha u(z)\, dz \right| \\ &\leq C \sum_{|\alpha|=m} \int_\Omega |x-z|^{-n+m} |D^\alpha u(z)|\, dz \\ &\leq C' d^{m-n/p} |u|_{W_p^m(\Omega)}. \end{aligned}$$

The proof can be completed via a density argument. □

Theorem 13.2 (xxx)**.** *If $1 < p < \infty$ and $m > n/p$, or $p = 1$ and $m \geq n$, there exist a constant $C = C(m, n, \gamma, p)$, such that*

$$\|R^m u\|_{L^\infty(\Omega)} \leq C d^{m-n/p} |u|_{W_p^m(\Omega)}$$

for all $u \in W_p^m(\Omega)$.

Proof of xxx. First, we assume that $u \in C^m(\Omega) \cap W_p^m(\Omega)$. We can use the pointwise representation of $R^m u(x)$.

$$\begin{aligned} |R^m u(x)| &= m \left| \sum_{|\alpha|=m} \int_{C_x} k_\alpha(x,z) D^\alpha u(z)\, dz \right| \\ &\leq C \sum_{|\alpha|=m} \int_\Omega |x-z|^{-n+m} |D^\alpha u(z)|\, dz \\ &\leq C' d^{m-n/p} |u|_{W_p^m(\Omega)}. \end{aligned}$$

The proof can be completed via a density argument. □

Theorem. *If $1 < p < \infty$ and $m > n/p$, or $p = 1$ and $m \geq n$, there exist a constant $C = C(m, n, \gamma, p)$, such that*

$$\|R^m u\|_{L^\infty(\Omega)} \leq C d^{m-n/p} |u|_{W_p^m(\Omega)}$$

for all $u \in W_p^m(\Omega)$.

Proof. First, we assume that $u \in C^m(\Omega) \cap W_p^m(\Omega)$. We can use the pointwise representation of $R^m u(x)$.

$$\begin{aligned} |R^m u(x)| &= m \left| \sum_{|\alpha|=m} \int_{C_x} k_\alpha(x,z) D^\alpha u(z)\, dz \right| \\ &\leq C \sum_{|\alpha|=m} \int_\Omega |x-z|^{-n+m} |D^\alpha u(z)|\, dz \\ &\leq C' d^{m-n/p} |u|_{W_p^m(\Omega)}. \end{aligned}$$

The proof can be completed via a density argument. □

Proposition 13.3.

$$Q^m u(x) = \sum_{|\lambda|<m} \left(\int_B \psi_\lambda(y) u(y)\, dy \right) x^\lambda$$

where $\psi_\lambda \in C_0^\infty(\mathbb{R}^n)$ and $\operatorname{supp}(\phi_\lambda) \in \overline{B}$.

Proof. This follows from xxx if we define

$$\psi_\lambda(y) = \sum_{\alpha \geq \lambda, |\alpha|<m} \frac{(-1)^{|\alpha|}}{\alpha!} a_{[\lambda,\alpha-\lambda]} D^\alpha (y^{\alpha-\lambda} \phi(y)).$$

□

Proposition 13.4 (xxx)**.**

$$Q^m u(x) = \sum_{|\lambda|<m} \left(\int_B \psi_\lambda(y) u(y)\, dy \right) x^\lambda$$

where $\psi_\lambda \in C_0^\infty(\mathbb{R}^n)$ *and* $\operatorname{supp}(\phi_\lambda) \in \overline{B}$.

Proof of xxx. This follows from xxx if we define

$$\psi_\lambda(y) = \sum_{\alpha \geq \lambda, |\alpha|<m} \frac{(-1)^{|\alpha|}}{\alpha!} a_{[\lambda,\alpha-\lambda]} D^\alpha (y^{\alpha-\lambda}\phi(y)).$$

□

Proposition.

$$Q^m u(x) = \sum_{|\lambda|<m} \left(\int_B \psi_\lambda(y) u(y)\, dy \right) x^\lambda$$

where $\psi_\lambda \in C_0^\infty(\mathbb{R}^n)$ *and* $\operatorname{supp}(\phi_\lambda) \in \overline{B}$.

Proof. This follows from xxx if we define

$$\psi_\lambda(y) = \sum_{\alpha \geq \lambda, |\alpha|<m} \frac{(-1)^{|\alpha|}}{\alpha!} a_{[\lambda,\alpha-\lambda]} D^\alpha (y^{\alpha-\lambda}\phi(y)).$$

□

14 Corollary, Lemma, Claim

Corollary 14.1. *Under the assumption of xxx, the following inequality holds*

$$\inf_{v \in P^{m-1}} \|u - v\|_{W_p^k(\Omega)} \leq C_{m,n,\gamma} d^{m-k} |u|_{W_p^k(\Omega)}, \quad k = 0, 1, \ldots, m,$$

Corollary 14.2 (xxx)**.** *Under the assumption of xxx, the following inequality holds*

$$\inf_{v \in P^{m-1}} \|u - v\|_{W_p^k(\Omega)} \leq C_{m,n,\gamma} d^{m-k} |u|_{W_p^k(\Omega)}, \quad k = 0, 1, \ldots, m,$$

Corollary. *Under the assumption of xxx, the following inequality holds*

$$\inf_{v \in P^{m-1}} \|u - v\|_{W_p^k(\Omega)} \leq C_{m,n,\gamma} d^{m-k} |u|_{W_p^k(\Omega)}, \quad k = 0, 1, \ldots, m,$$

Lemma 14.3. *Let* $f \in L^p(\Omega)$ *for* $p \geq 1$ *and* $m \geq 1$ *and let*

$$g(x) = \int_\Omega |x - z|^{-n+m} |f(z)|\, dz$$

Then

$$\|g\|_{L^p(\Omega)} \leq C_{m,n} d^m \|f\|_{L^p(\Omega)}.$$

Lemma 14.4 (xxx). *Let $f \in L^p(\Omega)$ for $p \geq 1$ and $m \geq 1$ and let*

$$g(x) = \int_\Omega |x - z|^{-n+m} |f(z)| \, dz$$

Then

$$\|g\|_{L^p(\Omega)} \leq C_{m,n} d^m \|f\|_{L^p(\Omega)}.$$

Lemma. *Let $f \in L^p(\Omega)$ for $p \geq 1$ and $m \geq 1$ and let*

$$g(x) = \int_\Omega |x - z|^{-n+m} |f(z)| \, dz$$

Then

$$\|g\|_{L^p(\Omega)} \leq C_{m,n} d^m \|f\|_{L^p(\Omega)}.$$

Claim 14.5. *$Q^m u$ is a polynomial of degree less than m in x.*

Claim 14.6 (xxx). *$Q^m u$ is a polynomial of degree less than m in x.*

Claim. *$Q^m u$ is a polynomial of degree less than m in x.*

15 Definition

Definition 15.1. Ω is star-shaped with respect to the ball B if , for all $x \in \Omega$, the closed convex hull of $\{x\} \cup B$ is a subset of Ω.

Definition 15.2 (xxx). Ω is star-shaped with respect to the ball B if , for all $x \in \Omega$, the closed convex hull of $\{x\} \cup B$ is a subset of Ω.

Definition. Ω is star-shaped with respect to the ball B if , for all $x \in \Omega$, the closed convex hull of $\{x\} \cup B$ is a subset of Ω.

16 Example

Example 16.1. The integral form of the Taylor remainder for $f \in C^m([0, 1])$ is given by

$$f(s) = \sum_{k=0}^{m-1} \frac{1}{k!} f^{(k)}(0) + \int_0^s \frac{1}{(m-1)!} f^{(m)}(t)(s-t)^{m-1} \, dt.$$

Example 16.2 (xxx). The integral form of the Taylor remainder for $f \in C^m([0, 1])$ is given by

$$f(s) = \sum_{k=0}^{m-1} \frac{1}{k!} f^{(k)}(0) + \int_0^s \frac{1}{(m-1)!} f^{(m)}(t)(s-t)^{m-1} \, dt.$$

Example. The integral form of the Taylor remainder for $f \in C^m([0,1])$ is given by

$$f(s) = \sum_{k=0}^{m-1} \frac{1}{k!} f^{(k)}(0) + \int_0^s \frac{1}{(m-1)!} f^{(m)}(t)(s-t)^{m-1}\, dt.$$

17 Problem, Solution

Problem 17.1. Calculate the integral of the function $g(x) = 3x^2$ with respect to x.

Solution 17.1. To calculate the integral of $g(x) = 3x^2$, we use the power rule for integration:

$$\int 3x^2\, dx = x^3 + C$$

where C is the constant of integration. □

Problem 17.2 (xxx)**.** Calculate the integral of the function $g(x) = 3x^2$ with respect to x.

Solution 17.2 (xxx)**.** To calculate the integral of $g(x) = 3x^2$, we use the power rule for integration:

$$\int 3x^2\, dx = x^3 + C$$

where C is the constant of integration. □

Problem. Calculate the integral of the function $g(x) = 3x^2$ with respect to x.

Solution. To calculate the integral of $g(x) = 3x^2$, we use the power rule for integration:

$$\int 3x^2\, dx = x^3 + C$$

where C is the constant of integration. □

18 Remark

Remark 18.1. Such a polynomial is not unique, due to the choice od cut-off function ϕ.

Remark 18.2 (xxx). Such a polynomial is not unique, due to the choice od cut-off function ϕ.

Remark. Such a polynomial is not unique, due to the choice od cut-off function ϕ.

19 Note

Note 19.1. The degree of $Q^m u$ is at most $m - 1$.

Note 19.2 (xxx). The degree of $Q^m u$ is at most $m - 1$.

Note. The degree of $Q^m u$ is at most $m - 1$.

参考文献

[1] OENCAN T, ALTINEL I K, LAPORTE G. A comparative analysis of several asymmetric traveling salesman problem formulations[J/OL]. Computers & Operations Research, 2009, 36(3): 637-654. DOI: 10.1016/j.cor.2007.11.008.

[2] ROBERTI R, TOTH P. Models and algorithms for the asymmetric traveling salesman problem: an experimental comparison[J/OL]. Euro Journal on Transportation & Logistics, 2012, 1(1-2): 113-133. DOI: 10.1007/s13676-012-0010-0.

[3] PAPADIMITRIOU C H, STEIGLITZ K. Combinatorial optimization: algorithms and complexity[M]. Dover edition ed. Mineola, NY: Dover Publications, 1998: 308-309.

[4] MILLER C E, TUCKER A W, ZEMLIN R A. Integer programming formulation of traveling salesman problems[J/OL]. Journal of the Acm, 1960, 7(4): 326-329. DOI: 10.1145/321043.321046.

Travelling Salesman Problem and Bellman-Held-Karp Algorithm

Quang Nhat Nguyen

May 10, 2020

1 Definitions

Definition 1.1 (Travelling Salesman Problem)**.** *Let $G = (V, E, \omega)$ be a* weighted *Hamiltonian graph, with $V = \{x_1, x_2, \ldots, x_n\}$, $i : E \to V \times V$ determines the edges, and $\omega : E \to \mathbb{R}_+$ determines the weighted edge lengths. The problem of finding its shortest Hamiltonian cycle is called the Travelling Salesman Problem (abbrv. TSP).*

Note: *If G is undirected, the TSP is called* symmetric TSP *(sTSP). If G is directed, the TSP is called* asymmetric TSP *(aTSP).*

This is one of the classical problems in Computer Science, and is an introductory problem to Dynamic Programming. The need of computer power arises when we consider a large graph (eg. one that contains hundreds or thousands of vertices) and want to find its shortest Hamiltonian cycle. Before discussing the approaches, let us introduce the *distance matrix.*

Definition 1.2 (Distance matrix)**.** *The $n \times n$ matrix D with elements*

$$D_{ij} := \min\{\omega(e) \,|\, e \in E,\, i(e) = (x_i, x_j)\} \quad \textit{if there exists such } e$$
$$D_{ij} := \infty \quad \textit{if there is no such } e$$

is called the distance matrix *of the weighted graph G.*

In simpler terms, the element D_{ij} denotes the shortest edge that originates at x_i and terminates at x_j. It can be seen that if the graph G is undirected, the matrix D is symmetric. If there is no edge from x_i to x_j, $D_{ij} = \infty$. If there are multiple edges from x_i to x_j, we only consider the shortest edge because our concern is the shortest Hamiltonian cycle.

A naïve approach to this problem would be for the computer to consider all permutations of the vertices, see if one permutation can make a cycle, and if it does then record the cycle's length for comparison. This method has a time complexity of $O(n!)$, and thus is not desirable when dealing with a large database.

Proposition 1.3. *One can reduce the time complexity of $O(n!)$ of the naïve method to $O(n^2 \times 2^n)$ by implementing Bellman-Held-Karp Algorithm.*

Let us discuss this algorithm in the next section.

2 Bellman-Held-Karp Algorithm

First, let us acknowledge the following.

Proposition 2.1. *The shortest Hamiltonian cycle does not depend on the choice of the starting vertex.*

This is obvious because the Hamiltonian cycle has cyclic symmetry, thus changing the starting vertex does not change the order of the other vertices in the cycle. Therefore, let us start constructing the desired path from x_1.

Definition 2.2. *Let $S \subset V\backslash\{x_1\} \equiv \{x_2, x_3, \ldots, x_n\}$ be a subset of size s $(1 \leq s \leq n-1)$. For each vertex $x_i \in S$, we define* $\text{cost}(x_i, S)$ *as the length of the shortest path from x_1 to x_i which travels to each of the remaining vertices in S once. More precisely, we implement this function by the following recursion formula:*

$$\text{cost}(x_i, S) = \min_{x_j}\{\text{cost}(x_j, S\backslash\{x_i\}) + D_{ji}\} \tag{2.1}$$

in which $x_j \in S\backslash\{x_i\}$. In case S has size 1, we define:

$$\text{cost}(x_i, S) = D_{1i} \tag{2.2}$$

Using the above recursive formula, we can gradually construct the cost function for subset S of size from 1 to $n-1$. Because of this recursive feature, which means that the current step is the base for the next step, the implementation of this algorithm in a programming language is called Dynamic Programming.

When we reach subset S of size $n-1$, which means $S = V\backslash\{x_1\}$, the only thing left is to find:

$$\begin{aligned}\text{Shortest Hamiltonian cycle} &= \min_{x_i}\{\text{cost}(x_i, S) + D_{i1}\} \\ &\equiv \min_{x_i}\{\text{cost}(x_i, V\backslash\{x_1\}) + D_{i1}\}\end{aligned}$$

with $x_i \in S \equiv V\backslash\{x_1\}$.

Proposition 2.3 (Time complexity of this algorithm)**.** *The time complexity of this algorithm is: $O(n^2 \times 2^n)$.*

Proof. This algorithm considers 2^{n-1} subsets of $V\backslash\{x_1\}$. For each subset, one computes the cost function for all of its elements, which there are no more than n of them. For each execution of the cost function, one again computes no more than n values based on the values obtained from the previous step. In total, the time complexity of this algorithm is: $O(n^2 \times 2^n)$. □

Remark 2.4. *Compared to the naïve method which has time complexity $O(n!)$, the time complexity of this algorithm, $O(n^2 \times 2^n)$, is much better. For example, if $n = 100$:*

$$\begin{aligned}O(n!) = O(100!) &= O(9.33 \times 10^{157}) \\ O(n^2 \times 2^n) = O(100^2 \times 2^{100}) &= O(1.27 \times 10^{34})\end{aligned}$$

which is about 7.35×10^{123}, or 0.7 septillion googols, times faster.

3 Illustration

Let us illustrate the algorithm through the following sample problem.

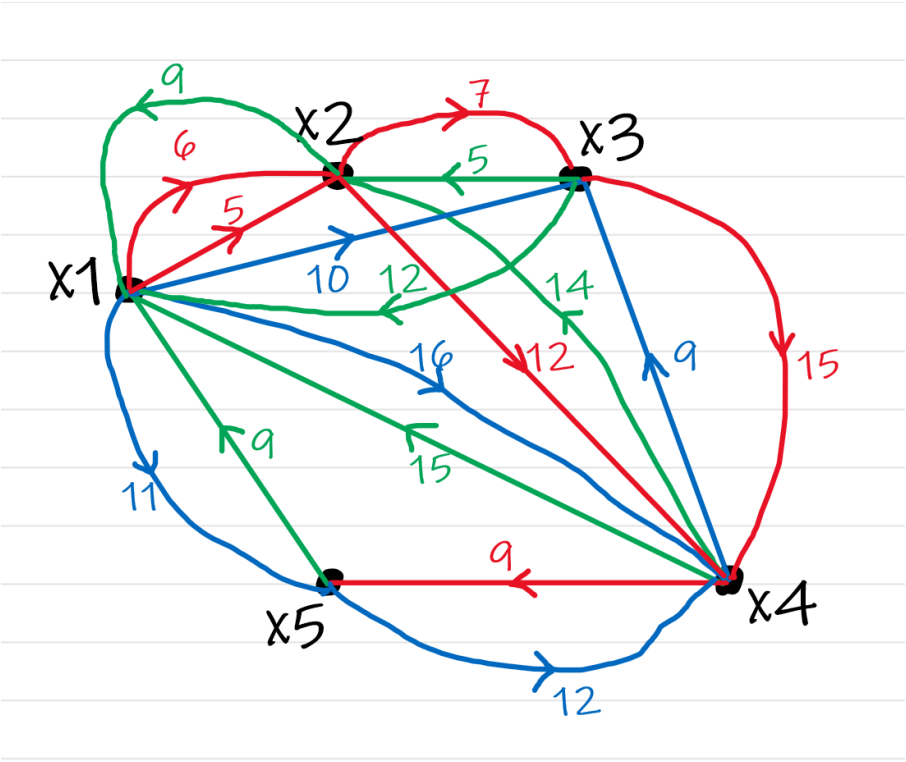


Figure 1: Graph for a sample asymmetric TSP

The distance matrix can be generated as follows:

$$\begin{bmatrix} 0 & 5 & 10 & 16 & 11 \\ 9 & 0 & 7 & 12 & \infty \\ 12 & 5 & 0 & 15 & \infty \\ 15 & 14 & 9 & 0 & 9 \\ 9 & \infty & \infty & 12 & 0 \end{bmatrix}$$

Now that we have the distance matrix, we can proceed with the recursive formula. First, let us consider subsets S of size 1:

Subset S	cost function	Result
$S = \{x_2\}$	$\text{cost}(x_2, \{x_2\}) = D_{12}$	5
$S = \{x_3\}$	$\text{cost}(x_3, \{x_3\}) = D_{13}$	10
$S = \{x_4\}$	$\text{cost}(x_4, \{x_4\}) = D_{14}$	16
$S = \{x_5\}$	$\text{cost}(x_5, \{x_5\}) = D_{15}$	11

Table 1: Process for subsets S of size 1

Next we consider subsets S of size 2 as follows:

Subset S	cost function	Result
$S = \{x_2, x_3\}$	$\text{cost}(x_2, \{x_2, x_3\}) = \min\{\text{cost}(x_3, \{x_3\}) + D_{32}\} = 10 + 5$	15
	$\text{cost}(x_3, \{x_2, x_3\}) = \min\{\text{cost}(x_2, \{x_2\}) + D_{23}\} = 5 + 7$	12
$S = \{x_2, x_4\}$	$\text{cost}(x_2, \{x_2, x_4\}) = \min\{\text{cost}(x_4, \{x_4\}) + D_{42}\} = 16 + 14$	30
	$\text{cost}(x_4, \{x_2, x_4\}) = \min\{\text{cost}(x_2, \{x_2\}) + D_{24}\} = 5 + 12$	17
$S = \{x_2, x_5\}$	$\text{cost}(x_2, \{x_2, x_5\}) = \min\{\text{cost}(x_5, \{x_5\}) + D_{52}\} = 11 + \infty$	∞
	$\text{cost}(x_5, \{x_2, x_5\}) = \min\{\text{cost}(x_2, \{x_2\}) + D_{25}\} = 5 + \infty$	∞
$S = \{x_3, x_4\}$	$\text{cost}(x_3, \{x_3, x_4\}) = \min\{\text{cost}(x_4, \{x_4\}) + D_{43}\} = 16 + 9$	25
	$\text{cost}(x_4, \{x_3, x_4\}) = \min\{\text{cost}(x_3, \{x_3\}) + D_{34}\} = 10 + 15$	25
$S = \{x_3, x_5\}$	$\text{cost}(x_3, \{x_3, x_5\}) = \min\{\text{cost}(x_5, \{x_5\}) + D_{53}\} = 11 + \infty$	∞
	$\text{cost}(x_5, \{x_3, x_5\}) = \min\{\text{cost}(x_3, \{x_3\}) + D_{35}\} = 10 + \infty$	∞
$S = \{x_4, x_5\}$	$\text{cost}(x_4, \{x_4, x_5\}) = \min\{\text{cost}(x_5, \{x_5\}) + D_{54}\} = 11 + 12$	23
	$\text{cost}(x_5, \{x_4, x_5\}) = \min\{\text{cost}(x_4, \{x_4\}) + D_{45}\} = 16 + 9$	25

Table 2: Process for subsets S of size 2

For the subsets S of size 3, we have the following table:

Subset S	cost function	Result
$S = \{x_2, x_3, x_4\}$	$\text{cost}(x_2, \{x_2, x_3, x_4\}) = \min\{25 + 5, 25 + 14\}$	30
	$\text{cost}(x_3, \{x_2, x_3, x_4\}) = \min\{30 + 7, 17 + 9\}$	26
	$\text{cost}(x_4, \{x_2, x_3, x_4\}) = \min\{15 + 12, 12 + 15\}$	27
$S = \{x_2, x_3, x_5\}$	$\text{cost}(x_2, \{x_2, x_3, x_5\}) = \min\{\infty, \infty\}$	∞
	$\text{cost}(x_3, \{x_2, x_3, x_5\}) = \min\{\infty, \infty\}$	∞
	$\text{cost}(x_5, \{x_2, x_3, x_5\}) = \min\{\infty, \infty\}$	∞
$S = \{x_2, x_4, x_5\}$	$\text{cost}(x_2, \{x_2, x_4, x_5\}) = \min\{23 + 14, \infty\}$	37
	$\text{cost}(x_4, \{x_2, x_4, x_5\}) = \min\{\infty, \infty\}$	∞
	$\text{cost}(x_5, \{x_2, x_4, x_5\}) = \min\{\infty, 17 + 9\}$	26
$S = \{x_3, x_4, x_5\}$	$\text{cost}(x_3, \{x_3, x_4, x_5\}) = \min\{23 + 9, 25 + \infty\}$	32
	$\text{cost}(x_4, \{x_3, x_4, x_5\}) = \min\{\infty, \infty\}$	∞
	$\text{cost}(x_5, \{x_3, x_4, x_5\}) = \min\{25 + \infty, 25 + 9\}$	34

Table 3: Process for subsets S of size 3

Note: Exlanation for the first entry:

$$\begin{aligned}\text{cost}(x_2, \{x_2, x_3, x_4\}) &= \min\{\text{cost}(x_3, \{x_3, x_4\}) + D_{32}, \text{cost}(x_4, \{x_3, x_4\}) + D_{42}\} \\ &= \min\{25 + 5, 25 + 14\} = 30\end{aligned}$$

For the subsets S of size 4, which means $S = \{x_2, x_3, x_4, x_5\}$, we have the following:

cost function	Evaluation	Result
$\text{cost}\{x_2, \{x_2, x_3, x_4, x_5\}\}$	$\min\{\text{cost}(x_3, \{x_3, x_4, x_5\}) + D_{32},$ $\text{cost}(x_4, \{x_3, x_4, x_5\}) + D_{42}, \quad = \min\{32+5, \infty, \infty\}$ $\text{cost}(x_5, \{x_3, x_4, x_5\}) + D_{52}, \}$	37
$\text{cost}\{x_3, \{x_2, x_3, x_4, x_5\}\}$	$\min\{\text{cost}(x_2, \{x_2, x_4, x_5\}) + D_{23},$ $\text{cost}(x_4, \{x_2, x_4, x_5\}) + D_{43}, \quad = \min\{37+7, \infty, \infty\}$ $\text{cost}(x_5, \{x_2, x_4, x_5\}) + D_{53}, \}$	44
$\text{cost}\{x_4, \{x_2, x_3, x_4, x_5\}\}$	$\min\{\text{cost}(x_2, \{x_2, x_3, x_5\}) + D_{24},$ $\text{cost}(x_3, \{x_2, x_3, x_5\}) + D_{34}, \quad = \min\{\infty, \infty, \infty\}$ $\text{cost}(x_5, \{x_2, x_3, x_5\}) + D_{54}, \}$	∞
$\text{cost}\{x_5, \{x_2, x_3, x_4, x_5\}\}$	$\min\{\text{cost}(x_2, \{x_2, x_3, x_4\}) + D_{25},$ $\text{cost}(x_3, \{x_2, x_3, x_4\}) + D_{35}, \quad = \min\{\infty, \infty, 27+9\}$ $\text{cost}(x_4, \{x_2, x_3, x_4\}) + D_{45}, \}$	36

Table 4: Process for subsets S of size 4

For the final step, we have:

$$\begin{aligned}\text{Shortest Hamiltonian cycle} = \min\{&\text{cost}\{x_2, \{x_2, x_3, x_4, x_5\}\} + D_{21},\\ &\text{cost}\{x_3, \{x_2, x_3, x_4, x_5\}\} + D_{31},\\ &\text{cost}\{x_4, \{x_2, x_3, x_4, x_5\}\} + D_{41},\\ &\text{cost}\{x_5, \{x_2, x_3, x_4, x_5\}\} + D_{51}\}\\ = \min\{&37 + 9, 44 + 12, \infty, 36 + 9\}\\ = 45\end{aligned}$$

This result means that this graph has *at least* three Hamiltonian cycles, and the shortest one has length 45. To trace back the order of the vertices in the shortest one, one simply trace back the vertices x_i in the minimal cost functions $\text{cost}(x_i, S)$. In this sample problem, the back-tracing order will be:

$$x_1 \leftarrow x_5 \leftarrow x_4 \leftarrow x_2 \leftarrow x_3 \leftarrow x_1$$

However, this back-tracing order is also correct:

$$x_1 \leftarrow x_5 \leftarrow x_4 \leftarrow x_3 \leftarrow x_2 \leftarrow x_1$$

So, the shortest Hamiltonian cycle has the following forward-going order:

$$x_1 \rightarrow x_3 \rightarrow x_2 \rightarrow x_4 \rightarrow x_5 \rightarrow x_1$$

or:

$$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow x_5 \rightarrow x_1$$

One can easily look at Figure 1 and check that the above Hamiltonian cycles both have length 45. This concludes the illustration of Bellman-Held-Karp Algorithm, which is based on Dynamic Programming